



MODUL AJAR DASAR PYTHON DENGAN GAME

**PyroRescue: Pengembangan Game Edukasi Python
Berbasis Web dengan Tema Kebakaran Hutan Kalimantan**

Disusun Oleh:

Muhammad Iqbal Ramadhan
Muhammad Indra

2410131210025
2410131210009

Jurusan Pendidikan Komputer
Fakultas Keguruan dan Ilmu Pendidikan
Universitas Lambung Mangkurat
2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena modul ajar Dasar Python dengan Game Edukasi PyroRescue dapat disusun sebagai salah satu bagian dari pengembangan media pembelajaran berbasis web. Modul ini dirancang untuk membantu pengguna yang baru mengenal pemrograman Python agar dapat mempelajari konsep melalui konteks yang sama dengan pengalaman di dalam permainan.

PyroRescue menggunakan misi penanganan kebakaran hutan Kalimantan sebagai lingkungan belajar. Pengguna berperan sebagai petugas pemadam yang bergerak menuju sumber air, menyiapkan persediaan, melakukan penyemprotan berulang, dan mengambil keputusan untuk menyelamatkan bekantan setelah area dinyatakan aman. Setiap tindakan tersebut dihubungkan dengan konsep Python yang dipelajari secara bertahap: variabel pada Level 1, perulangan for pada Level 2, serta percabangan if/else pada Level 3.

Modul ini tidak hanya memberikan definisi dan contoh kode. Setiap bab disusun menyerupai alur permainan: pengguna membaca situasi, memahami tujuan misi, mempelajari konsep yang diperlukan, menuliskan kode pada simulasi PyroPad, memprediksi dampak kode terhadap dunia game, memeriksa umpan balik, lalu menyelesaikan challenge dan evaluasi. Dengan pola ini, pengguna diharapkan tidak sekadar menghafal sintaks, tetapi memahami alasan suatu kode digunakan dan akibatnya terhadap keadaan permainan.

Penyusun menyadari modul ini masih dapat dikembangkan. Kritik dan saran sangat diharapkan untuk penyempurnaan materi, tampilan, dan keterhubungan antara modul dengan mekanik permainan PyroRescue.

Banjarmasin, 25 September 2026

Tim Pengembang PyroRescue

DAFTAR ISI

- Identitas Modul
- Kata Pengantar
- Petunjuk Penggunaan Modul
- Pendahuluan: Belajar Python melalui PyroRescue
- Peta Konsep
- BAB I - Level 1: Variabel dan Persiapan Air
- BAB II - Level 2: Perulangan For dan Pemadaman Api
- BAB III - Level 3: If/Else dan Penyelamatan Bekantan
- BAB IV - Integrasi Konsep dan Evaluasi Akhir
- Glosarium
- Daftar Pustaka
- Lampiran Kunci Jawaban dan Ketercapaian Tujuan Pembelajaran

PETUNJUK PENGGUNAAN MODUL

Modul PyroRescue digunakan seperti sebuah latihan permainan di atas kertas. Bacalah skenario misi secara berurutan, lalu bayangkan bahwa setiap kotak kode merupakan area PyroPad tempat pengguna menulis instruksi. Setelah kode dituliskan, jangan langsung melihat pembahasan. Prediksi terlebih dahulu apa yang akan terjadi pada karakter, persediaan air, titik api, atau status bekantan.

1. Mulai dari Level 1. Materi disusun progresif sehingga konsep yang dipelajari pada level sebelumnya akan digunakan kembali pada level berikutnya.
2. Pada bagian “Situasi Misi”, pahami keadaan awal permainan dan target yang harus dicapai.
3. Pada bagian “Belajar Konsep”, baca penjelasan sampai memahami fungsi setiap bagian kode.
4. Pada bagian “Simulasi PyroPad”, tuliskan atau lengkapi kode. Setelah itu, bandingkan prediksi dengan “Respons Game”.
5. Jika jawaban belum tepat, baca “Feedback” dan “Hint” secara bertahap. Hint pertama tidak langsung memberikan jawaban lengkap.
6. Selesaikan Challenge 1, Challenge 2, Challenge 3, dan Evaluasi Level sebelum melanjutkan ke level berikutnya.
7. Gunakan bagian Rangkuman dan Refleksi untuk memastikan bahwa kamu memahami hubungan kode dengan tindakan di dalam game.

CATATAN PENTING TENTANG PERINTAH GAME

Perintah maju(n), mundur(n), kanan(n), kiri(n), dan semprot() adalah fungsi/perintah yang dibuat khusus untuk PyroRescue. Penulisannya mengikuti bentuk pemanggilan fungsi Python, tetapi fungsi-fungsi tersebut bukan fungsi bawaan Python standar. Materi utama yang dinilai adalah variabel, for/range(), Boolean, serta if/else.

PENDAHULUAN

BELAJAR PYTHON MELALUI PYRORESCUE

Pemrograman adalah proses menyusun instruksi agar komputer melakukan langkah tertentu untuk menyelesaikan masalah. Dalam game PyroRescue, instruksi tersebut tidak berhenti sebagai teks. Kode yang ditulis pengguna mengubah keadaan permainan: karakter dapat bergerak, tangki air dapat terisi, penyemprotan dapat dilakukan berulang, api dapat padam, dan status penyelamatan bekantan dapat berubah.

Cara belajar di PyroRescue mengikuti prinsip dari sederhana menuju kompleks. Pada Level 1, pengguna belajar bahwa data dapat disimpan dan diubah menggunakan variabel. Pada Level 2, pengguna belajar bahwa tindakan yang sama tidak perlu ditulis berkali-kali karena dapat diulang dengan `for` dan `range()`. Pada Level 3, pengguna belajar bahwa program dapat mengambil keputusan berbeda berdasarkan kondisi `True` atau `False` melalui `if` dan `else`.

Tujuan Pembelajaran Umum

Setelah mempelajari modul dan menyelesaikan rangkaian misi PyroRescue, pengguna diharapkan mampu memahami serta menerapkan konsep dasar Python untuk menyelesaikan masalah sederhana yang disajikan dalam bentuk misi permainan.

- Menjelaskan fungsi variabel sebagai tempat menyimpan data dan menggunakan operator penugasan (`=`).
- Membuat, memperbarui, dan menggunakan nilai dari variabel lain.
- Menggunakan `for` dan `range()` untuk menjalankan aksi berulang dalam jumlah tertentu.
- Memahami Boolean `True` dan `False` sebagai representasi kondisi.
- Menggunakan `if` dan `else` untuk menentukan tindakan berdasarkan kondisi.
- Menggabungkan variabel, perulangan, dan percabangan dalam penyelesaian satu misi lengkap.

Peta Konsep

ALUR PEMBELAJARAN

LEVEL 1 — Variabel → menyimpan dan mengubah persediaan air

LEVEL 2 — Variabel + `for/range()` → mengulang `semprot()` untuk memadamkan api

LEVEL 3 — Variabel + `for/range()` + `if/else` → memeriksa kondisi api dan menentukan status bekantan

HASIL AKHIR — pengguna mampu membaca situasi, memilih konsep, menulis kode, melihat akibat, dan memperbaiki kesalahan.

BAB I

LEVEL 1: VARIABEL DAN PERSIAPAN AIR

TUJUAN PEMBELAJARAN

Setelah menyelesaikan bab ini, kamu mampu membuat variabel, memberikan nilai, mengubah nilai yang tersimpan, menggunakan nilai dari variabel lain, serta menghubungkan perubahan nilai dengan persediaan air di dalam PyroRescue.

1.1 Apersepsi - Misi Pertama Dimulai

Kebakaran mulai menyebar di kawasan hutan. Sebelum menuju titik api, seorang petugas tidak dapat berangkat dengan tangki kosong. Ia harus menuju pompa air di tepi sungai, menentukan berapa unit air yang akan dibawa, kemudian memastikan persediaan tersebut tersedia pada pos pemadaman. Dalam game, komputer perlu “mengingat” jumlah air itu. Data tersebut disimpan menggunakan variabel.

TAHUKAH KAMU?

Game membutuhkan banyak data yang terus berubah. Posisi pemain, jumlah air, status api, dan status bekantan semuanya dapat direpresentasikan sebagai data. Variabel memberi nama pada data sehingga program dapat menggunakannya kembali.

1.2 Tutorial Navigasi PyroPad

Sebelum belajar variabel, kenali empat perintah gerak yang digunakan untuk memindahkan karakter di peta. Angka di dalam tanda kurung menunjukkan jumlah petak yang ditempuh. Pada permainan, titik merah berkedip menunjukkan lokasi tujuan.

```
maju(1)
mundur(1)
kanan(1)
kiri(1)
```

Contoh berikut memerintahkan karakter bergerak maju tiga petak, kemudian bergerak ke kanan dua petak.

```
maju(3)
kanan(2)
```

COBA PREDIKSI

Jika karakter berada di titik awal dan kode maju(2) lalu kanan(1) dijalankan, gambarkan atau bayangkan posisi akhir karakter. Tujuan latihan ini bukan menilai konsep variabel, tetapi membiasakan kamu membaca urutan instruksi.

1.3 Apa Itu Variabel?

Variabel dapat dibayangkan sebagai wadah yang diberi label. Labelnya adalah nama variabel, sedangkan isi wadahnya adalah nilai. Pada PyroRescue, tangki air adalah analogi yang mudah: nama variabel menyatakan data persediaan, sedangkan angka menyatakan banyaknya air.

```
isi_air = 3
```

Kode tersebut dibaca “variabel `isi_air` diberi nilai 3”. Tanda sama dengan (=) pada baris tersebut merupakan operator penugasan atau assignment. Artinya, nilai di sebelah kanan disimpan ke nama variabel di sebelah kiri.

Bagian kode	Makna
<code>isi_air</code>	Nama variabel
<code>=</code>	Operator penugasan
<code>3</code>	Nilai yang disimpan

1.4 Aturan Dasar Penamaan Variabel

Python memiliki aturan dalam memberi nama variabel. Aturan ini perlu diperhatikan karena kesalahan kecil pada nama dapat membuat program gagal atau mengakses data yang berbeda.

- Nama dapat terdiri atas huruf, angka, dan garis bawah (_).
- Nama tidak boleh diawali angka.
- Nama tidak boleh mengandung spasi.
- Kata kunci Python seperti `if`, `else`, dan `for` tidak boleh digunakan sebagai nama variabel.
- Python membedakan huruf besar dan kecil. `isi_air` dan `Isi_Air` dianggap berbeda.
- Gunakan nama yang menjelaskan isi data agar kode mudah dibaca.

1.5 Nilai Variabel Dapat Berubah

Satu variabel dapat diberi nilai baru. Ketika sebuah assignment baru dijalankan, nilai sebelumnya pada variabel tersebut digantikan oleh nilai terbaru.

```
isi_air = 3  
isi_air = 5
```

Setelah kedua baris selesai dijalankan, nilai akhir `isi_air` adalah 5. Dalam PyroRescue, perubahan ini dapat divisualisasikan pada HUD sebagai AIR 3 berubah menjadi 5.

1.6 Menggunakan Nilai dari Variabel Lain

Nilai yang telah disimpan dapat diberikan kepada variabel lain. Inilah alasan variabel berguna: program tidak selalu harus menuliskan angka yang sama berulang kali.

```
isi_air = 5
air_pos = isi_air
```

Pada baris kedua, Python membaca nilai yang saat itu tersimpan pada `isi_air`, yaitu 5, kemudian menyimpannya ke `air_pos`. Setelah baris tersebut dijalankan, `isi_air` bernilai 5 dan `air_pos` juga bernilai 5.

1.7 Challenge 1 - Mengambil Air

SITUASI MISI

Karakter telah tiba di pompa air di tepi sungai. Kondisi awal tangki adalah `isi_air = 0`. Pos pemadaman meminta 3 unit air. Tugasmu adalah menuliskan kode yang membuat persediaan air menjadi 3 unit.

Di dalam game, kamu akan melihat PyroPad. Tuliskan satu baris kode untuk menyimpan 3 ke dalam variabel `isi_air`.

```
isi_air = ...
```

Prediksi sebelum membuka pembahasan

Apa yang akan berubah setelah kode benar dijalankan? Tuliskan prediksimu: apakah posisi karakter berubah, api langsung padam, atau nilai AIR pada HUD berubah?

RESPONS GAME

Jika kode `isi_air = 3` benar, pompa bergerak, air keluar, tangki terisi, dan HUD berubah dari AIR 0 menjadi AIR 3. Game menampilkan: “✓ Berhasil! Variabel `isi_air` sekarang menyimpan nilai 3.”

JIKA NILAI SALAH

Misalnya kamu menulis `isi_air = 2`. Program dapat membaca sintaksnya, tetapi target misi belum terpenuhi. Feedback: “Persediaan belum sesuai. Target: 3. Nilai `isi_air`: 2.”

1.8 Challenge 2 - Memperbarui Persediaan di Pos 1

SITUASI MISI

Persediaan sekarang 3 unit, sedangkan kebutuhan Pos 1 adalah 5 unit. Kamu tidak perlu membuat variabel baru. Perbarui nilai `isi_air` agar menjadi 5.

```
isi_air = ...
```

Saat kode yang benar dijalankan, nilai lama digantikan. Perhatikan bahwa variabelnya tetap bernama `isi_air`; hanya nilainya yang berubah.

RESPONS GAME

HUD menampilkan AIR 3 → 5. Feedback: “✓ Berhasil! Nilai `isi_air` diperbarui menjadi 5.”

1.9 Challenge 3 - Memberikan Air ke Pos 1

SITUASI MISI

Kondisi awal: `isi_air = 5` dan `air_pos = 0`. Pos 1 harus menerima jumlah air yang sama dengan persediaan pemain. Jangan menulis angka 5 lagi. Gunakan nilai dari variabel `isi_air`.

```
air_pos = ...
```

Kode yang diharapkan adalah `air_pos = isi_air`. Ini menunjukkan bahwa sebuah variabel dapat menggunakan nilai yang tersimpan pada variabel lain.

RESPONS GAME

Visual menunjukkan aliran persediaan dari pemain ke Pos 1. `air_pos` berubah menjadi 5. Sistem game dapat mengosongkan `isi_air` setelah penyerahan sebagai bagian dari mekanik; pemain tidak perlu menulis `isi_air = 0` pada challenge ini.

1.10 Evaluasi Level 1

Sekarang bantuan langkah demi langkah dikurangi. Skenario baru: Pos berikutnya membutuhkan 4 unit air. Siapkan persediaan pemain, kemudian salin persediaan tersebut ke pos menggunakan nilai variabel, bukan dengan menulis angka 4 dua kali.

```
# Tulis jawabanmu di sini
```

KRITERIA KEBERHASILAN

Program dianggap benar jika nilai akhir `isi_air` = 4 dan `air_pos` = 4, serta `air_pos` memperoleh nilai melalui `isi_air`. Setelah evaluasi berhasil, game membuka Level 2.

1.11 Kesalahan Umum dan Cara Membacanya

Kode/Keadaan	Masalah	Perbaikan
<code>isi_air =</code>	Kode belum lengkap	Berikan nilai setelah tanda <code>=</code> .
<code>isi_air = 2</code> saat target 3	Sintaks benar, tetapi nilai target salah	Baca kembali kebutuhan misi.
<code>1air = 3</code>	Nama variabel diawali angka	Gunakan nama seperti <code>isi_air</code> .
<code>air_pos = isi_air</code> sebelum <code>isi_air</code> dibuat	Variabel sumber belum memiliki nilai	Buat <code>isi_air</code> terlebih dahulu.

Rangkuman Level 1

Pada Level 1 kamu belajar bahwa variabel adalah nama yang digunakan untuk menyimpan data. Assignment dengan tanda `=` memberikan nilai kepada variabel. Nilai yang sama dapat diperbarui dengan assignment baru, dan nilai suatu variabel dapat digunakan untuk mengisi variabel lain. Di PyroRescue, konsep tersebut dipakai untuk mengelola persediaan air.

Refleksi

1. Jelaskan dengan bahasamu sendiri arti `isi_air = 3`.
2. Mengapa `isi_air = 3` lalu `isi_air = 5` menghasilkan nilai akhir 5?
3. Apa keuntungan menulis `air_pos = isi_air` dibanding `air_pos = 5` jika nilai `isi_air` suatu saat berubah?
4. Bagian mana yang masih membingungkan: nama variabel, tanda `=`, atau penggunaan nilai variabel lain?

Kuis BAB I

1. Kode yang tepat untuk menyimpan 3 unit air adalah ...

- A. `3 = isi_air`
- B. `isi_air = 3`
- C. `isi air = 3`
- D. `isi_air == 3`

2. Setelah menjalankan `isi_air = 3` kemudian `isi_air = 5`, nilai `isi_air` adalah ...

- A. 3
- B. 5
- C. 8
- D. Tidak ada nilai

3. Jika `isi_air = 5`, hasil `air_pos = isi_air` adalah ...

- A. `air_pos = 0`
- B. `air_pos = 5`
- C. `isi_air = 0` otomatis oleh Python
- D. error

4. Nama variabel yang valid adalah ...

- A. 1air
- B. isi air
- C. if
- D. jumlah_semprot

5. Tanda `=` pada `isi_air = 4` berfungsi untuk ...

- A. membandingkan dua nilai
- B. memberikan nilai
- C. mengulang kode
- D. menjalankan kondisi

BAB II

LEVEL 2: PERULANGAN FOR DAN PEMADAMAN API

TUJUAN PEMBELAJARAN

Setelah menyelesaikan bab ini, kamu mampu menjelaskan mengapa perulangan dibutuhkan, menggunakan `for` dan `range()`, menulis indentasi dengan benar, serta menggunakan variabel untuk menentukan jumlah aksi `semprot()` yang dilakukan.

2.1 Apersepsi - Satu Api, Banyak Semprotan

Setelah persediaan air siap, karakter memasuki area kebakaran. Satu titik api kecil mungkin padam dengan satu semprotan, tetapi api yang lebih besar membutuhkan beberapa semprotan. Menulis perintah yang sama berkali-kali memang dapat menghasilkan aksi yang sama, tetapi program menyediakan cara yang lebih ringkas dan terstruktur: perulangan.

```
semprot()  
semprot()  
semprot()
```

Tiga baris di atas menghasilkan tiga aksi semprot. Namun pada Level 2, target belajar bukan sekadar membuat api padam. Kamu harus menyatakan pola “ulangi aksi ini tiga kali” menggunakan `for loop`.

```
for i in range(3):  
    semprot()
```

2.2 Mengenal Perintah `semprot()`

Dalam `PyroRescue`, `semprot()` adalah fungsi khusus game. Satu kali pemanggilan berarti satu kali aksi penyemprotan: karakter mengangkat selang, air keluar, kekuatan api berkurang, dan persediaan air berkurang sesuai aturan game.

```
semprot()
```

ATURAN KHUSUS PYRORESCUE

`semprot()` dirancang tanpa argumen. Artinya, `semprot(3)` tidak digunakan untuk tiga kali semprotan. Untuk mengulangi aksi, gunakan `for` dan `range()`.

2.3 Bentuk Dasar For Loop

```
for i in range(3):  
    semprot()
```

Baris pertama merupakan header pengulangan. Kata `for` menandai bahwa blok akan diulang. Nama `i` adalah variabel pengontrol. `range(3)` menghasilkan tiga iterasi. Tanda titik dua (`:`) membuka blok. Baris `semprot()` menjorok ke kanan karena termasuk blok yang diulang.

2.4 Memahami range()

Untuk kebutuhan awal PyroRescue, bentuk `range(n)` dapat dipahami sebagai “ulangi sebanyak `n` kali”. `range(1)` menghasilkan satu iterasi, `range(2)` dua iterasi, dan seterusnya. Secara internal, `range(3)` menghasilkan nilai 0, 1, dan 2 untuk variabel pengontrol, tetapi yang paling penting pada tahap ini adalah jumlah iterasinya tetap tiga.

Kode	Jumlah iterasi	Dampak di game
<code>range(1)</code>	1	<code>semprot()</code> 1 kali
<code>range(2)</code>	2	<code>semprot()</code> 2 kali
<code>range(3)</code>	3	<code>semprot()</code> 3 kali
<code>range(5)</code>	5	<code>semprot()</code> 5 kali

2.5 Mengapa Indentasi Penting?

Python menggunakan indentasi untuk menentukan baris mana yang termasuk ke dalam sebuah blok. Pada `for` loop, aksi yang akan diulang harus ditulis menjorok setelah tanda titik dua.

BENAR

```
for i in range(3):  
    semprot()
```

SALAH

```
for i in range(3):  
semprot()
```

Pada contoh salah, `semprot()` tidak berada di dalam blok `for`. Karena itu, struktur kode tidak sesuai dengan tujuan pengulangan.

2.6 Menggunakan Variabel sebagai Jumlah Perulangan

Konsep Level 1 tidak ditinggalkan. Jumlah iterasi dapat disimpan terlebih dahulu pada sebuah variabel, kemudian variabel tersebut dipakai oleh range().

```
jumlah_semprot = 4

for i in range(jumlah_semprot):
    semprot()
```

Ketika range(jumlah_semprot) dijalankan, Python membaca nilai jumlah_semprot. Jika nilainya 4, blok semprot() dijalankan empat kali. Jika nilainya diubah menjadi 2, loop hanya berjalan dua kali.

2.7 Challenge 1 - Mengenal Semprotan

SITUASI MISI

Api kecil di depan jalur hanya membutuhkan satu penyemprotan. Jalankan satu aksi semprot().

```
# Tulis satu perintah di sini
```

RESPONS GAME

Karakter mengangkat selang → air keluar → api mengecil/padam → air berkurang 1. Feedback: “✓ semprot() menjalankan satu kali aksi penyemprotan.”

2.8 Challenge 2 - Mengulang Semprotan

SITUASI MISI

Api berikutnya membutuhkan tepat 3 kali penyemprotan. Challenge meminta penggunaan perulangan, bukan menulis semprot() tiga kali.

```
for i in range(...):  
    semprot()
```

Jika kamu menulis `semprot()` tiga kali, aksi mungkin tetap terjadi, tetapi sistem pembelajaran memberikan feedback bahwa challenge meminta konsep `for`. Artinya, game tidak hanya menilai hasil akhir, tetapi juga strategi kode yang digunakan.

RESPONS GAME

Iterasi 1: api besar → sedang. Iterasi 2: api sedang → kecil. Iterasi 3: api kecil → padam. Feedback: “✓ Berhasil! `range(3)` membuat aksi diulang 3 kali.”

2.9 Challenge 3 - Variabel dalam Perulangan

SITUASI MISI

Titik api ketiga membutuhkan 4 semprotan. Simpan jumlah tersebut pada variabel `jumlah_semprot`, kemudian gunakan variabel tersebut di dalam `range()`.

```
jumlah_semprot = ...
```

```
for i in range(...):  
    semprot()
```

RESPONS GAME

`jumlah_semprot` dibaca sebagai 4 → loop berjalan 4 kali → `semprot()` dijalankan empat kali → api padam. Jika `jumlah_semprot = 2`, feedback menunjukkan “Api belum padam. Dibutuhkan: 4. Dilakukan: 2.”

2.10 Evaluasi Level 2

Titik api terakhir membutuhkan 5 kali semprotan. Tulis program yang menggunakan sebuah variabel `jumlah_semprot` dan `for` loop. Jangan menulis `semprot()` lima kali secara manual.

```
# Tulis jawabanmu di sini
```

KRITERIA KEBERHASILAN

Ada variabel jumlah_semprot dengan nilai 5; menggunakan for; menggunakan range(); semprot() berada di dalam loop; dan api berhasil padam.

2.11 Kesalahan Umum dan Feedback

Kesalahan	Dampak	Perbaikan
for i in range(3) tanpa :	Header belum lengkap	Tambahkan titik dua.
semprot() tidak diindentasi	Blok loop tidak terbentuk dengan benar	Indentasikan semprot().
range(2) saat target 4	Aksi kurang	Gunakan jumlah iterasi sesuai kebutuhan.
semprot(3)	Tidak sesuai kontrak fungsi PyroRescue	Gunakan for i in range(3): semprot().

2.12. Sistem Hint Bertahap

Saat pemain mengalami kesulitan, PyroRescue tidak langsung menampilkan jawaban lengkap. Hint diberikan bertahap supaya pemain masih melakukan proses berpikir.

1. Hint 1: Pikirkan bagaimana mengulang satu aksi tanpa menulisnya berkali-kali.
2. Hint 2: Gunakan for dan range() untuk menentukan jumlah pengulangan.
3. Hint 3: Bentuk kode: for i in range(...): lalu letakkan semprot() di dalam blok dengan indentasi.

Rangkuman Level 2

For loop digunakan untuk mengulang sebuah blok. range(n) menentukan jumlah iterasi pada contoh dalam modul ini. Indentasi menunjukkan aksi mana yang diulang. Nilai n tidak harus ditulis langsung; nilai tersebut dapat berasal dari variabel jumlah_semprot. Dalam PyroRescue, pola ini digunakan agar satu perintah semprot() dapat dijalankan beberapa kali secara terstruktur.

14. Refleksi

1. Mengapa menulis semprot() tiga kali berbeda tujuan belajarnya dengan menggunakan for i in range(3)?
 2. Apa yang terjadi jika jumlah_semprot bernilai 2 tetapi api membutuhkan 4 semprotan?
 3. Mengapa indentasi diperlukan pada semprot() di bawah for?
 4. Apakah range(3) berarti nilai i adalah 1,2,3? Jelaskan berdasarkan pembahasan modul.
- Modul Pemahaman PyroRescue

Kuis BAB II

1. Kode yang menjalankan `semprot()` tiga kali adalah ...
 - A. `semprot(3)`
 - B. `for i in range(3):` lalu `semprot()` terindentasi
 - C. `range = 3`
 - D. `if semprot()`
2. Fungsi `range(4)` pada contoh modul adalah ...
 - A. membuat 4 variabel
 - B. menghasilkan 4 iterasi
 - C. mengisi air 4 unit
 - D. mematikan api otomatis
3. Mengapa `semprot()` ditulis menjorok?
 - A. agar menjadi komentar
 - B. agar termasuk blok `for`
 - C. agar menjadi variabel
 - D. tidak ada alasan
4. Jika `jumlah_semprot = 5`, maka `range(jumlah_semprot)` berarti ...
 - A. loop 0 kali
 - B. loop 1 kali
 - C. loop 5 kali
 - D. error
5. `semprot(3)` ditolak di `PyroRescue` karena ...
 - A. angka tidak ada di Python
 - B. `semprot()` dirancang tidak menerima argumen
 - C. `for` tidak boleh dipakai
 - D. api sudah padam

BAB III

LEVEL 3: IF/ELSE DAN PENYELAMATAN BEKANTAN

TUJUAN PEMBELAJARAN

Setelah menyelesaikan bab ini, kamu mampu memahami Boolean True/False, menggunakan if untuk menjalankan aksi saat kondisi benar, menggunakan else sebagai alternatif saat kondisi salah, dan menggabungkannya dengan variabel serta for loop dalam misi penyelamatan bekantan.

3.1 Apersepsi - Apakah Area Sudah Aman?

Di Level 3, misi tidak selesai hanya karena pemain mencapai suatu lokasi. Bekantan berada dekat area kebakaran dan hanya dapat dinyatakan aman setelah api dipadamkan. Program harus memeriksa keadaan dunia game terlebih dahulu. Jika api padam, penyelamatan dapat dilanjutkan. Jika api belum padam, bekantan harus menunggu. Inilah masalah yang diselesaikan dengan percabangan.

3.2 Boolean: True dan False

Boolean adalah tipe data dengan dua kemungkinan nilai: True dan False. Di PyroRescue, api_padam digunakan sebagai variabel kondisi. True berarti api telah padam, sedangkan False berarti api masih menyala.

```
api_padam = True
# atau
api_padam = False
```

PERHATIKAN

True dan False ditulis tanpa tanda petik. Jika kamu menulis "True", nilai tersebut menjadi teks (string), bukan Boolean.

3.3 Menggunakan if

If digunakan untuk menjalankan sebuah blok hanya ketika kondisi bernilai benar. Contoh berikut berarti: jika api_padam bernilai True, ubah status_bekantan menjadi "aman".

```
if api_padam:
    status_bekantan = "aman"
```

Jika `api_padam` bernilai `False`, baris di dalam blok `if` dilewati. Ini berbeda dari assignment `api_padam = True`. Assignment menyimpan kondisi, sedangkan `if` memeriksa kondisi.

3.4 Menggunakan `if/else`

Sering kali program membutuhkan dua jalur. Jika kondisi benar lakukan A, jika salah lakukan B. Struktur `if/else` menyediakan dua kemungkinan tersebut.

```
if api_padam:
    status_bekantan = "selamat"
else:
    status_bekantan = "tunggu"
```

Ketika `api_padam` `True`, blok `if` dijalankan dan `else` dilewati. Ketika `api_padam` `False`, blok `if` dilewati dan blok `else` dijalankan. Hanya salah satu jalur yang dipilih pada satu kali pemeriksaan.

3.5 Indentasi dalam Percabangan

Seperti `for` loop, `if` dan `else` menggunakan indentasi untuk menunjukkan blok. Semua perintah yang harus dijalankan di bawah kondisi harus menjorok dengan konsisten.

BENAR

```
if api_padam:
    status_bekantan = "selamat"
else:
    status_bekantan = "tunggu"
```

3.6 Kondisi dengan Perbandingan

Selain Boolean langsung, Python juga dapat menghasilkan kondisi menggunakan operator perbandingan. Bagian ini membantu kamu memahami bahwa kondisi dapat berasal dari data lain.

Operator	Arti
<code>==</code>	sama dengan
<code>!=</code>	tidak sama dengan
<code>></code>	lebih dari
<code><</code>	kurang dari
<code>>=</code>	lebih dari atau sama dengan
<code><=</code>	kurang dari atau sama dengan

```
isi_air = 3

if isi_air >= 3:
    status_persiapan = "cukup"
else:
    status_persiapan = "kurang"
```

Pada gameplay utama Level 3, kondisi dibuat lebih sederhana melalui `api_padam` agar fokus pengguna tetap pada konsep if/else. Namun contoh perbandingan ini menunjukkan bagaimana Boolean dapat dihasilkan dari data.

3.7 Challenge 1 - Memadamkan Api di Habitat

SITUASI MISI

Api di dekat habitat membutuhkan 3 kali semprotan. Tentukan jumlah semprotan dengan variabel, kemudian gunakan for. Setelah jumlah semprotan terpenuhi, sistem game mengubah `api_padam` menjadi True.

```
jumlah_semprot = ...

for i in range(jumlah_semprot):
    semprot()
```

RESPONS GAME

Jika semprotan cukup: api mengecil hingga padam → sistem memperbarui `api_padam = True`. Jika semprotan kurang: `api_padam` tetap False.

3.8 Challenge 2 - Memeriksa Kondisi Api dengan if

SITUASI MISI

Api telah dipadamkan. Sekarang program harus memeriksa `api_padam`. Jika benar, status bekantan diubah menjadi "aman".

```
if api_padam:
    status_bekantan = "..."
```

RESPONS GAME

Jika `api_padam` True, HUD menampilkan BEKANTAN: AMAN. Jika False, blok if tidak berjalan dan feedback menyatakan bahwa kondisi belum terpenuhi.

3.9 Challenge 3 - Menentukan Tindakan dengan if/else

SITUASI MISI

Game harus memiliki tindakan untuk kedua keadaan. Jika api padam, bekantan dinyatakan "selamat". Jika api belum padam, status menjadi "tunggu".

```
if api_padam:  
    status_bekantan = "..."  
else:  
    status_bekantan = "..."
```

Challenge ini menekankan bahwa else bukan kondisi baru. Else berarti “jika kondisi if tidak terpenuhi”.

3.10 Evaluasi Level 3 - Misi Penyelamatan Lengkap

Api terakhir membutuhkan 4 semprotan. Tugasmu adalah menentukan jumlah semprotan, menjalankan perulangan, lalu memeriksa kondisi api dan menentukan status bekantan. Ini merupakan integrasi materi Level 1, Level 2, dan Level 3.

```
# Tulis program lengkapmu di sini
```

KRITERIA KEBERHASILAN

Program memiliki jumlah_semprot = 4; menggunakan for dan range(jumlah_semprot); semprot() berada di dalam loop; menggunakan if api_padam; memiliki else; dan menghasilkan status_bekantan yang sesuai keadaan api.

3.11 Membaca Alur Eksekusi

ALUR JIKA BERHASIL

jumlah_semprot = 4 → for berjalan 4 kali → semprot() × 4 → api padam → api_padam = True → if api_padam dipilih → status_bekantan = "selamat" → misi selesai.

ALUR JIKA BELUM BERHASIL

jumlah_semprot terlalu kecil → api belum padam → api_padam = False → kondisi if tidak terpenuhi → else dijalankan → status_bekantan = "tunggu" → pemain perlu memperbaiki kode.

12. Kesalahan Umum dan Feedback

Kesalahan	Masalah	Perbaikan
if api_padam tanpa :	Header belum lengkap	Tambahkan titik dua.
api_padam = "True"	Nilai menjadi string	Gunakan True tanpa tanda petik.
if api_padam = True:	Assignment digunakan sebagai kondisi	Gunakan if api_padam: atau perbandingan == bila diperlukan.
Blok if/else tidak diindentasi	Struktur blok salah	Gunakan indentasi konsisten.

Rangkuman Level 3

Boolean memiliki dua nilai, True dan False. If memilih aksi ketika kondisi benar, sedangkan else menjalankan alternatif ketika kondisi if tidak terpenuhi. Nilai kondisi dapat disimpan dalam variabel seperti api_padam. Dalam PyroRescue, percabangan dipakai untuk menentukan apakah bekantan sudah dapat dinyatakan selamat atau masih harus menunggu.

Refleksi

1. Apa perbedaan api_padam = True dengan if api_padam:?
2. Mengapa status_bekantan membutuhkan if/else, bukan hanya satu assignment?
3. Apa yang terjadi jika api belum padam tetapi program tidak memiliki else?
4. Ceritakan dengan kalimatmu sendiri alur dari semprot() sampai status_bekantan = "selamat".

Kuis BAB III

1. Nilai Boolean yang menyatakan api belum padam adalah ...

- A. "False"
- B. False
- C. 0 selalu
- D. else

2. if api_padam: berfungsi untuk ...

- A. memberi nilai pada api_padam
- B. memeriksa kondisi api_padam
- C. mengulang semprot()
- D. membuat fungsi

3. Blok else dijalankan ketika ...

- A. kondisi if True
- B. kondisi if False
- C. range habis
- D. variabel dibuat

4. Kode yang benar adalah ...

- A. if api_padam = True:
- B. if api_padam:

- C. if = api_padam
- D. else api_padam

5. Jika api_padam = False, hasil struktur if/else pada contoh adalah ...

- A. status_bekantan = "selamat"
- B. status_bekantan = "tunggu"
- C. semprot() berjalan otomatis
- D. isi_air = 0

BAB IV

INTEGRASI KONSEP DAN EVALUASI AKHIR

4.1 Hubungan Level 1 sampai Level 3

Tiga level PyroRescue disusun sebagai satu jalur pembelajaran. Konsep baru tidak menggantikan konsep lama, tetapi menambah kemampuan pengguna. Level 1 membentuk dasar penyimpanan data. Level 2 memakai kembali variabel untuk mengatur jumlah pengulangan. Level 3 memakai variabel dan perulangan, kemudian menambahkan percabangan untuk menentukan keputusan.

Tahap	Masalah Game	Konsep Target	Contoh
Level 1	Menyiapkan dan menyerahkan air	Variabel	isi_air = 3
Level 2	Api membutuhkan penyemprotan berulang	Variabel + for/range	for i in range(jumlah_semprot)
Level 3	Penyelamatan bergantung pada kondisi api	Variabel + for + if/else	if api_padam: ... else: ...

4.2 Contoh Integrasi Tiga Konsep

```
jumlah_semprot = 4

for i in range(jumlah_semprot):
    semprot()

if api_padam:
    status_bekantan = "selamat"
else:
    status_bekantan = "tunggu"
```

Program di atas memiliki tiga lapisan logika. Pertama, variabel `jumlah_semprot` menyimpan data. Kedua, `for` loop menggunakan data tersebut untuk mengulang aksi. Ketiga, `if/else` memeriksa kondisi hasil dari tindakan dan menentukan status penyelamatan. Pola seperti ini adalah contoh sederhana bagaimana beberapa konsep pemrograman bekerja bersama dalam satu tujuan.

4.3 Input - Proses - Output dalam PyroRescue

Salah satu cara memahami program adalah memisahkan data masukan, proses, dan hasil. Pada PyroRescue, input dapat berupa nilai `isi_air`, `jumlah_semprot`, atau kondisi awal api. Proses berupa assignment, perulangan, dan percabangan. Output dapat terlihat sebagai perubahan HUD, animasi api, atau status bekantan.

4.4 Siklus Belajar di Dalam Game

SIKLUS PYRORESCUE

Baca misi → tentukan target → pilih konsep Python → tulis kode di PyroPad → tekan RUN → lihat animasi/perubahan state → baca feedback → perbaiki kode jika perlu → challenge selesai → checkpoint tersimpan.

Tombol RESET digunakan ketika pengguna ingin mengulang challenge dari checkpoint aktif. Reset mengembalikan posisi karakter, kondisi api, persediaan air yang relevan, objective, dan animasi ke keadaan awal challenge. Dengan demikian, pengguna dapat mencoba kembali tanpa harus mengulang seluruh level.

Evaluasi Akhir

Kerjakan tanpa melihat pembahasan sebelumnya. Evaluasi terdiri atas pilihan ganda dan tugas menulis kode. Tujuannya adalah memeriksa apakah kamu dapat memilih konsep yang tepat, bukan hanya menyalin bentuk kode.

1. Pernyataan `isi_air = 4` termasuk konsep apa?

A. Percabangan B. Assignment variabel C. Perulangan D. Boolean

2. Jika `isi_air = 3` kemudian `isi_air = 6`, nilai akhir `isi_air` adalah ...

A. 3 B. 6 C. 9 D. error

3. Kode `air_pos = isi_air` berarti ...

A. membandingkan dua variabel B. menyalin nilai `isi_air` ke `air_pos` C. menghapus `isi_air` D. mengulang `air_pos`

4. Perintah yang tepat untuk satu penyemprotan adalah ...

A. `semprot()` B. `semprot(1)` wajib C. `for semprot` D. `api_padam()`

5. `range(5)` pada materi modul digunakan untuk ...

A. lima kondisi B. lima iterasi C. lima variabel D. lima level

6. Mengapa `semprot()` harus diindentasi di bawah `for`?

A. menjadi bagian blok loop B. menjadi komentar C. menjadi string D. tidak perlu

7. Nilai Boolean yang benar adalah ...

A. "True" B. true C. True D. TRUE

8. `if api_padam:` akan menjalankan blok ketika ...

A. `api_padam` False B. `api_padam` True C. `jumlah_semprot` 0 D. selalu

9. else digunakan untuk ...

A. menyimpan data B. mengulang data C. alternatif ketika if tidak terpenuhi D. membuat fungsi

10. Tujuan penggunaan jumlah_semprom = 4 sebelum for adalah ...

A. menyimpan jumlah iterasi B. mematikan api tanpa loop C. membuat Boolean D. menggerakkan karakter

11. Tuliskan kode untuk menyimpan 5 unit air pada isi_air.

12. Tuliskan dua baris kode untuk membuat isi_air = 5 dan menyalin nilainya ke air_pos.

13. Tuliskan for loop yang menjalankan semprom() tiga kali.

14. Buat jumlah_semprom = 4 lalu gunakan dalam range().

15. Tuliskan api_padam = False dan jelaskan maknanya dalam game.

16. Tuliskan if yang membuat status_bekantan = "aman" jika api_padam benar.

17. Lengkapi struktur if/else agar status menjadi "selamat" jika api padam dan "tunggu" jika belum.

18. Analisis kode: jumlah_semprom = 2 lalu loop semprom(), sedangkan api membutuhkan 4 semprom. Apa yang terjadi dan mengapa?

19. Jelaskan perbedaan semprom() tiga kali secara manual dan for i in range(3): semprom() dari sisi tujuan pembelajaran.

20. Tulis program lengkap evaluasi penyelamatan: 4 semprom, kemudian tentukan status bekantan menggunakan if/else.

Refleksi Akhir

Setelah menyelesaikan modul, tinjau kembali cara berpikirmu. Apakah kamu sekarang dapat melihat bahwa kode adalah instruksi yang mengubah keadaan game? Apakah kamu dapat menentukan kapan cukup menggunakan variabel, kapan membutuhkan perulangan, dan kapan harus menggunakan percabangan? Tuliskan satu contoh misi baru yang menurutmu dapat dibuat dengan menggabungkan ketiga konsep tersebut.

GLOSARIUM

Istilah	Pengertian
Assignment	Pemberian nilai kepada variabel menggunakan tanda =.
Boolean	Tipe data dengan dua nilai: True dan False.
Challenge	Tahap misi yang menguji penerapan konsep tertentu.
Checkpoint	Titik penyimpanan kemajuan challenge aktif.
Condition/Kondisi	Nilai atau ekspresi yang dievaluasi sebagai benar atau salah.
For loop	Struktur untuk mengulang blok kode.
Indentasi	Spasi pada awal baris untuk menandai blok kode Python.
Iterasi	Satu kali putaran dalam proses pengulangan.
PyroPad	Area/perangkat kontrol di PyroRescue tempat pemain menulis dan menjalankan kode.
range()	Fungsi Python yang menghasilkan urutan nilai dan digunakan pada for loop.
State	Keadaan suatu objek/data pada game, misalnya api_padam atau jumlah air.
Variabel	Nama yang merujuk pada nilai/data yang disimpan dalam program.

DAFTAR PUSTAKA

- Gustiandi, B. (2023). Langkah Awal Menguasai Bahasa Pemrograman Python. Jakarta: Penerbit BRIN. ISBN 978-623-8372-21-8.
- Purba, H. S., Sukmawati, R. A., & Adini, M. H. (2021). Pemrograman Dasar Menggunakan Python. Yogyakarta: Deepublish (CV Budi Utama). ISBN 978-623-02-3507-8.

LAMPIRAN

Lampiran 1 — Kunci Kuis

Kuis 1: 1-B, 2-B, 3-B, 4-D, 5-B.

Kuis 2: 1-B, 2-B, 3-B, 4-C, 5-B.

Kuis 3: 1-B, 2-B, 3-B, 4-B, 5-B.

Lampiran 2 — Contoh Jawaban Evaluasi Level

Evaluasi Level 1

```
isi_air = 4
air_pos = isi_air
```

Evaluasi Level 2

```
jumlah_semprot = 5

for i in range(jumlah_semprot):
    semprot()
```

Evaluasi Level 3

```
jumlah_semprot = 4

for i in range(jumlah_semprot):
    semprot()

if api_padam:
    status_bekantan = "selamat"
else:
    status_bekantan = "tunggu"
```

Lampiran 3 — Kunci Evaluasi Akhir

Pilihan ganda: 1-B, 2-B, 3-B, 4-A, 5-B, 6-A, 7-C, 8-B, 9-C, 10-A. Nomor 11–20 merupakan tugas kode/uraian dan dinilai berdasarkan ketepatan sintaks, penggunaan konsep target, serta kesesuaian hasil dengan skenario.

Lampiran 4 — Ketercapaian Tujuan Pembelajaran

Tujuan Pembelajaran	Level/Bukti	Indikator Ketercapaian	Taksonomi
Membuat, mengubah, dan menggunakan nilai variabel	Level 1 Challenge 1–3, Evaluasi Level 1, Kuis 1	Pengguna menulis assignment dengan nama dan nilai yang benar serta mampu menggunakan nilai variabel lain.	C3 Menerapkan
Menggunakan for dan range()	Level 2 Challenge 2–3, Evaluasi Level 2, Kuis 2	Pengguna membentuk loop dengan jumlah iterasi sesuai target dan indentasi benar.	C3 Menerapkan
Menganalisis hubungan nilai variabel dengan jumlah iterasi	Level 2 Challenge 3	Pengguna menjelaskan bahwa perubahan jumlah_semprot mengubah banyaknya aksi semprot().	C4 Menganalisis

Menggunakan Boolean dan if/else	Level 3 Challenge 2–3, Kuis 3	Pengguna menentukan jalur tindakan berdasarkan True/False dan menulis blok if/else dengan benar.	C3 Menerapkan
Mengintegrasikan variabel, loop, dan percabangan	Evaluasi Level 3 dan Evaluasi Akhir	Pengguna menyusun program lengkap dari penentuan jumlah semprotan hingga status penyelamatan.	C4 Menganalisis